

Othello - a walkthrough

Othello

A board game on the GPU, which is mostly a bad idea, done anyway — and the useful part is knowing exactly which half of it was worth doing.

You are black, on green felt. Four computer opponents: two random-ish, one alpha-beta, and one that plays sixteen thousand random games in three milliseconds and picks whichever move won most. The last one is on the GPU. The other three are not, and *that is the point of the example*.

Source	examples/othello/ — three files, 1,000 lines
Board	two 64-bit words, no arrays
Players	Beginner, Intermediate, Advanced (alpha-beta 4-ply), Master (GPU)
Kernel	one, dispatched once per move
Threads per move	4,096 per candidate — about 40,000 for a typical position
Verified against	published perft counts to depth 7 (55,092)

These documents

Chapter	What it covers
1. Where the ideas come from	Norvig's array board, bitboards, alpha-beta, and how Monte-Carlo got respectable
2. Does a board game want a GPU?	The honest answer — mostly no — and the measurements behind both halves of it
3. The board is two integers	board.mojo: the whole ruleset as shifts and masks
4. Four players and one kernel	ai.mojo: the evaluation, the search, the playout, the dispatch
5. The window and the pump	main.mojo: why the event loop is hand-rolled, and the bug that taught it
6. What to understand	The things that will surprise you, and the ones that will bite
7. The sequel that does not work	The hybrid UCT spike, four fixed bugs, and why it still loses 0–6

The shortest possible summary

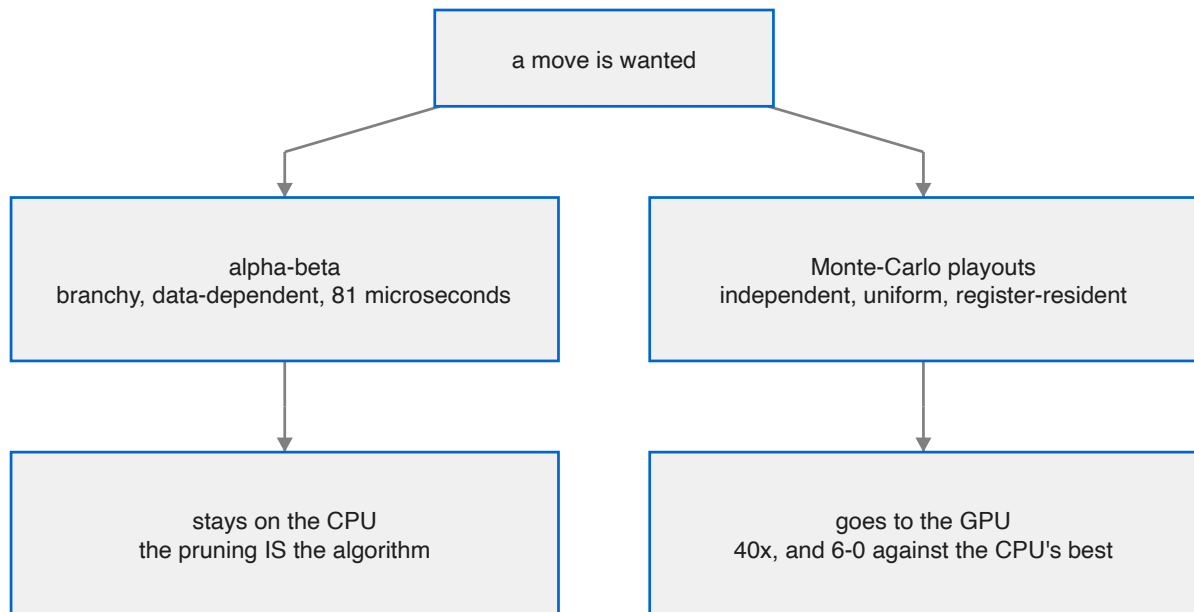
An Othello position is 64 squares. A machine word is 64 bits. So the position is **two words** — the squares you hold and the squares your opponent holds — and every rule in the game becomes shifts and masks over those two words.

That representation is what makes the rest possible. A single GPU thread can hold an entire game in two registers, which means it can play one out to the end without touching memory, which means ten thousand threads can each play a different game at once and never talk to each other.

That is the *only* part of a board game shaped like a GPU problem. The strongest classical technique — alpha-beta — is shaped like the exact opposite: its whole advantage is skipping work based on what it has already found, and threads that skip different work diverge. So the example ships alpha-beta on the CPU and Monte-Carlo on the GPU, and says so.

The measurements are the argument:

alpha-beta, 4 ply	81 µs on the CPU — nothing to accelerate
16,384 playouts	136.8 ms CPU vs 3.4 ms GPU — 40×
Master (GPU) vs Advanced (alpha-beta)	6–0 , margins of 6 to 26 discs



1. Where the ideas come from

Every technique in this example is older than the hardware it runs on, and two of them are much older. The source comments name their ancestry directly — *"Norvig's version in Paradigms of AI Programming"*, *"Norvig's weighted squares"* — so the lineage is worth following, because it explains why the code makes the choices it does.

The game

Othello — Reversi, renamed and trademarked in 1971 — is played on 8×8 with discs that are black on one side and white on the other. You place a disc so that it brackets an unbroken line of your opponent's discs against another of yours, and every disc in that line turns over. The player with more discs at the end wins.

Three properties matter for what follows, and all three are unusual:

- **The board never empties.** Every move adds a disc; the game is over in at most 60 moves. There is a hard, short horizon.
- **Ownership is violently unstable.** A single move can flip a dozen discs. Counting material mid-game tells you almost nothing — a player ahead 40–4 can lose.
- **Corners are permanent.** A disc in a corner can never be bracketed, so it can never be flipped. Everything else is negotiable.

That last point is the whole of classical Othello strategy, and it is why the evaluation function in `ai.mojo` looks the way it does.

Norvig, and the board this example does *not* use

Peter Norvig's *Paradigms of Artificial Intelligence Programming* (1992) devotes a chapter to Othello, and it is the direct ancestor of this example — the port started from a Common Lisp demo in that tradition.

Norvig represents the board as a **100-element array**: 64 playing squares inside a border of sentinel values. The border exists so that a walk off the edge hits a wall rather than wrapping onto the opposite rank. The source is explicit that this is the right choice *for its language*:

That is the right shape for a language with arrays and no wide integers, and it costs a bounds check per step in eight directions.

Two things carry over from Norvig unchanged. The first is the **weighted squares table** — corners at 120, the squares diagonally beside them at –40, edges modest, the middle nearly worthless. The second is the observation that a table is not enough, which is why `smart_weighted` adds a bonus for corners actually held.

What does *not* carry over is the array.

Bitboards

A board is 64 squares. A machine word is 64 bits. Chess programmers noticed this in the 1970s — 64 squares there too — and built a whole discipline around representing positions as bit-parallel sets rather than arrays of pieces.

Othello is the game bitboards fit best, because Othello's rules are *already* set operations. "Every square where a line of opponent discs is bracketed by one of mine" is not a loop; it is a shifted, masked propagation:

```
var run = shift(own, dir) & opp
for _ in range(5):
    run |= shift(run, dir) & opp
moves |= shift(run, dir) & empty
```

Norvig's sentinel border becomes a mask applied after the shift. Eight directions, no bounds checks, no branches that depend on the position.

The source is clear that this is not a micro-optimisation:

That matters twice over here. It is faster on the CPU, and it is the reason a GPU player is possible at all: a thread that plays a whole game needs two registers for the board, and every thread executes the same instructions regardless of what its game looks like.

Hold on to that sentence. It is the hinge the entire example turns on, and [chapter 2](#) is about nothing else.

Minimax and alpha-beta

The classical answer to "what should I play?" is a tree search, and its history is short and definitive.

Minimax applied to games is Claude Shannon's, from his 1950 paper on programming a computer to play chess: assume your opponent plays the move worst for you, recurse, back the value up.

Alpha-beta is the observation that most of that tree never needs to be looked at. If one reply already refutes a move, the *other* replies to that move cannot make it better — so stop. It was found independently by several people in the late 1950s and 1960s, and given its definitive analysis by Knuth and Moore in 1975. Done well, it examines roughly the square root of the nodes minimax would.

`negamax` in `ai.mojo` is textbook alpha-beta, in the negamax formulation that collapses the two cases into one by negating on the way up. It handles the two Othello-specific wrinkles that a chess implementation would not have:

```
if moves == 0:
    if legal_moves(opp, own) == 0:
        # the game is over -- count discs, do not consult the table
        # Pass: the turn goes over, the depth does not.
        return -negamax(opp, own, depth, -beta, -alpha, smart)
```

A player with no legal move **passes** rather than losing, and a pass must not consume depth or a long forced sequence would be searched to nothing.

And the machine that ended the argument

Othello is a solved argument in a way chess is not. Michael Buro's **Logistello** beat the reigning world champion Takeshi Murakami **6–0** in 1997, and it did so with a hand-built, statistically fitted pattern evaluation searching deeply — not with more randomness. Strong Othello has been a solved-in-practice engineering problem for nearly thirty years.

In 2023 it was pushed further: 8×8 Othello was weakly solved, and perfect play from the standard opening is a **draw**.

This matters to the example in a slightly deflating way. Nothing here is near the state of the art, and nothing here is trying to be. The question the example answers is not "how strong can Othello get?" but "which parts of a board game are shaped like GPU work?" — and that question is worth asking precisely because a game this well-understood makes the answer checkable.

How Monte-Carlo got respectable

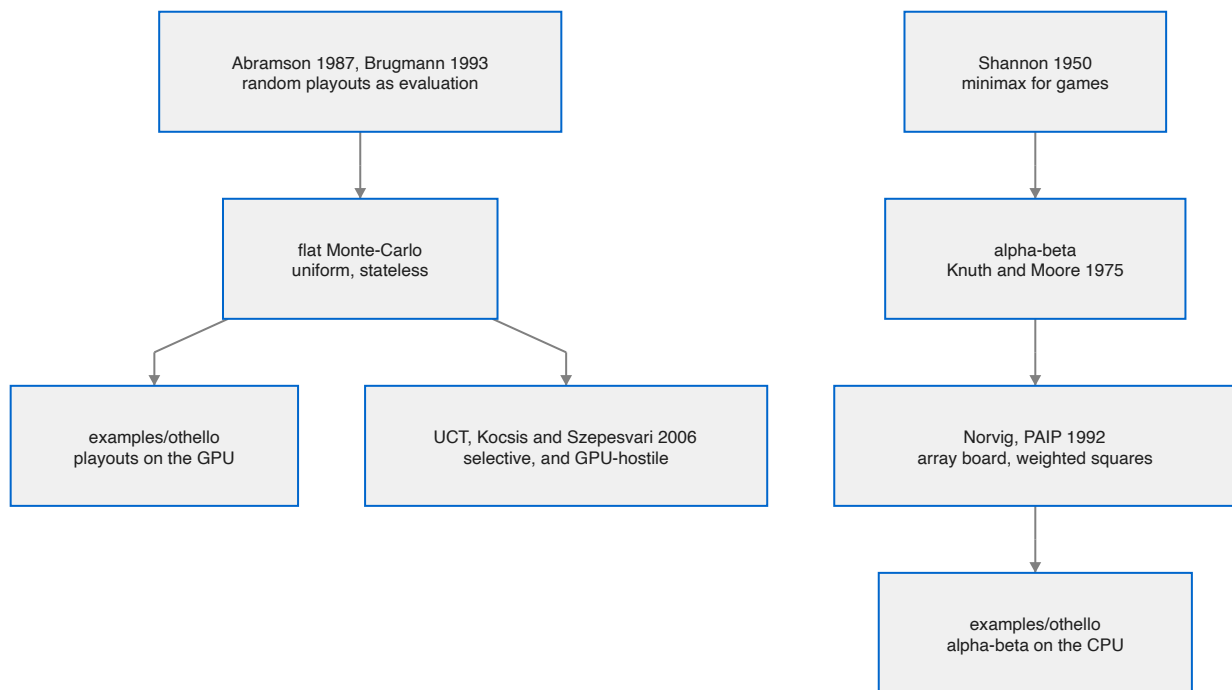
For most of that history, "play random games and count the wins" was not a serious proposal. It became one in stages:

- **1987** — Bruce Abramson's thesis argues that the *expected outcome under random play* is a usable evaluation function, which was not obvious.
- **1993** — Bernd Brügmann's *Monte Carlo Go* plays it straight: no evaluation function at all, just random playouts. It plays badly, but it plays, in a game where handcrafted evaluation had stalled.
- **2006** — two papers land in the same year. Rémi Coulom names and formalises Monte-Carlo Tree Search; Kocsis and Szepesvári derive **UCT**, which applies a bandit algorithm (UCB1, Auer *et al.* 2002) to deciding which branch to sample next. Suddenly the samples go where they matter.
- **2016** — AlphaGo, which is MCTS with the random playout replaced by a neural network.

The example's Master player is deliberately from the **first** stage of that story, not the last. It is *flat* Monte-Carlo: samples spread evenly over the root moves, results discarded after each decision, no tree at all.

That is the crudest member of the family, and it is on the GPU **because** it is crude — uniform sampling has no shared state, so ten thousand threads never need to agree about anything. The spike in [spikes/othello-mcts/](#) is the attempt to have UCT's selectivity *and* the GPU's throughput, and [chapter 7](#) is the honest account of how that went.

The lineage



2. Does a board game want a GPU?

This is the chapter the example exists for. The source states the question in its first paragraph and does not soften the answer:

A port of a Common Lisp demo, and an excuse to answer a question honestly: where does a GPU help a board game, and where does it not?

The answer is **mostly no**. What follows is the case against, then the narrow case for, then why the narrow case was worth building anyway.

Five reasons this is a bad idea

1. There is nothing to accelerate

The strongest CPU player in the app searches four ply of alpha-beta. Measured on the machine this was written on:

depth	time
3	22 μ s
4	81 μs
6	1,009 μ s

Eighty-one microseconds. A human cannot perceive it; a display cannot show it. The source draws the obvious conclusion and refuses the obvious temptation:

Eighty-one microseconds. Moving that to a GPU would make it slower, and saying otherwise would be a demo rather than an argument.

That is the first and most important objection, and it is not about algorithms. **You cannot speed up something that has already finished.** A GPU would add a kernel launch, a buffer, and a synchronise to a problem measured in tens of microseconds — the overhead alone exceeds the work.

2. The best algorithm for the job is actively hostile to the hardware

Alpha-beta's entire advantage is *not looking*. When one reply already refutes a move, the remaining replies to that move cannot change the answer, so they are skipped. A well-ordered search examines about the square root of the nodes that plain minimax would.

Now put that on a GPU. Threads in a warp execute together; when they take different branches, the hardware runs both sides and masks off the threads that should not be executing. A search where **every thread prunes somewhere different** is the worst possible case for that: near-total divergence, all the time.

You can avoid the divergence, and the way to avoid it is to stop pruning — to have every thread search the same shape of tree. But the pruning *was the algorithm*. You would be running exponentially more nodes in order to run them in parallel. The source puts it in one line:

Threads would diverge on the first cutoff, and keeping them in step means giving up the pruning, which was the entire point.

There is a deeper version of this objection. Alpha-beta is not just branchy — it is **sequentially dependent**. The window that makes a cutoff possible comes from results already computed. Search two siblings genuinely in parallel and neither can narrow the other's window, so both do more work than either would have done alone. Parallel alpha-beta is a real and difficult research area for exactly this reason, and none of its answers look like a GPU.

3. The algorithm that *does* fit is the weak one

Flat Monte-Carlo — sample every root move uniformly, keep the winner — fits a GPU beautifully. It is also, as the spike README says without flinching, *"the crudest member of its family"*.

It spreads samples evenly over moves that are obviously bad and obviously good alike, and it discards everything it learned the moment the move is played. UCT — the version that made Monte-Carlo famous and eventually beat professional Go players — is enormously stronger per sample, and it is GPU-hostile for the *same reason alpha-beta is*:

the tree is shared mutable state, and what to sample next depends on what the last sample found. The same tension that rules out alpha-beta.

So the GPU does not merely fail to help the good algorithm. **It selects for a worse one.** Choosing an inferior method because it parallelises is one of the oldest traps in the subject, and this example walks straight into it on purpose, with a note explaining that it has.

4. A game wants latency; a GPU sells throughput

A board game needs **one** answer, **now**. That is a latency problem.

A GPU is a throughput machine: it is spectacular at doing forty thousand independent things and mediocre at doing one thing quickly. Every move here pays a fixed toll — compile lookup, buffer allocation, dispatch, `synchronize`, map back to the host — regardless of how much thinking happened in between. For alpha-beta at 81 μ s that toll is the entire cost. It is only bearable because a playout player *has* forty thousand independent things.

5. Strength does not come from randomness

Logistello beat the human world champion 6–0 in 1997 with a fitted pattern evaluation and a deep search. The strongest Othello programs are still built that way. None of them got there by playing more random games.

A GPU buys you *more samples*. Othello strength is bought with a *better evaluation function*. Those are different currencies, and the exchange rate is poor.

The one place it genuinely fits

Set all of that aside and look at what a single playout actually does:

```
fn playout(black_in: UInt64, white_in: UInt64, black_to_move: Bool,
  seed: UInt64) -> Int:
  """Play to the end at random. Returns +1 if black wins, -1 white, 0 drawn.

  The same function runs on the CPU and inside the GPU kernel. It touches
  no memory, which is why ten thousand copies of it can run at once.
  """
```

Check it against what the hardware wants, property by property:

What a GPU wants	What a playout does
independence	every game is separate; no thread reads another's anything
no memory traffic	the whole state is two UInt64s and an RNG — registers
uniform control flow	every thread runs the same loop: legal moves, pick one, flip
arithmetic density	shifts, masks, popcounts, sixty times over
a large batch	4,096 playouts × ~10 candidate moves = ~40,000 threads
a tiny result	one Int32 per thread

That is not a workload that has been *made* to fit a GPU. It is a workload that already was one, and the only reason it fits is the board representation. Norvig's 100-element array would put a per-thread array in memory, and forty thousand threads each walking their own array is a different and much worse program.

The measurement, 16,384 playouts from the opening position:

CPU	136.8 ms
GPU	3.4 ms
	40×

Both pick the same move, which is the cheap correctness check worth doing whenever the same algorithm exists twice.

So why build it?

Six reasons, in rough order of how much they justify the effort.

1. To be able to say "no" credibly. A claim that alpha-beta does not want a GPU is worth nothing from someone who never tried. The example ships both, on the same board code, with numbers. The 81 μ s is what makes the refusal an argument rather than an opinion.

2. Because speed here buys strength, not just speed. This is the part that is easy to miss. Forty times faster does not mean the same player answers sooner — it means a *different player becomes affordable*. A level that would stall for most of a second per move can now answer while your hand is still on the mouse, so it can afford to be the strongest thing in the app:

```
Master (GPU playouts) 6 - 0 Advanced (alpha-beta, 4 ply)
```

with margins from 6 to 26 discs, and all six games played in half a second. Crude-but-plentiful beat sophisticated-but-shallow. That is a genuine result and not a foregone one.

3. Because it forced the right representation. The bitboard was adopted so *that* a thread could hold a game. It then turned out to make the CPU players faster and the ruleset shorter — 120 lines for the entire game, verified against published perft counts. The constraint improved the code it was imposed on.

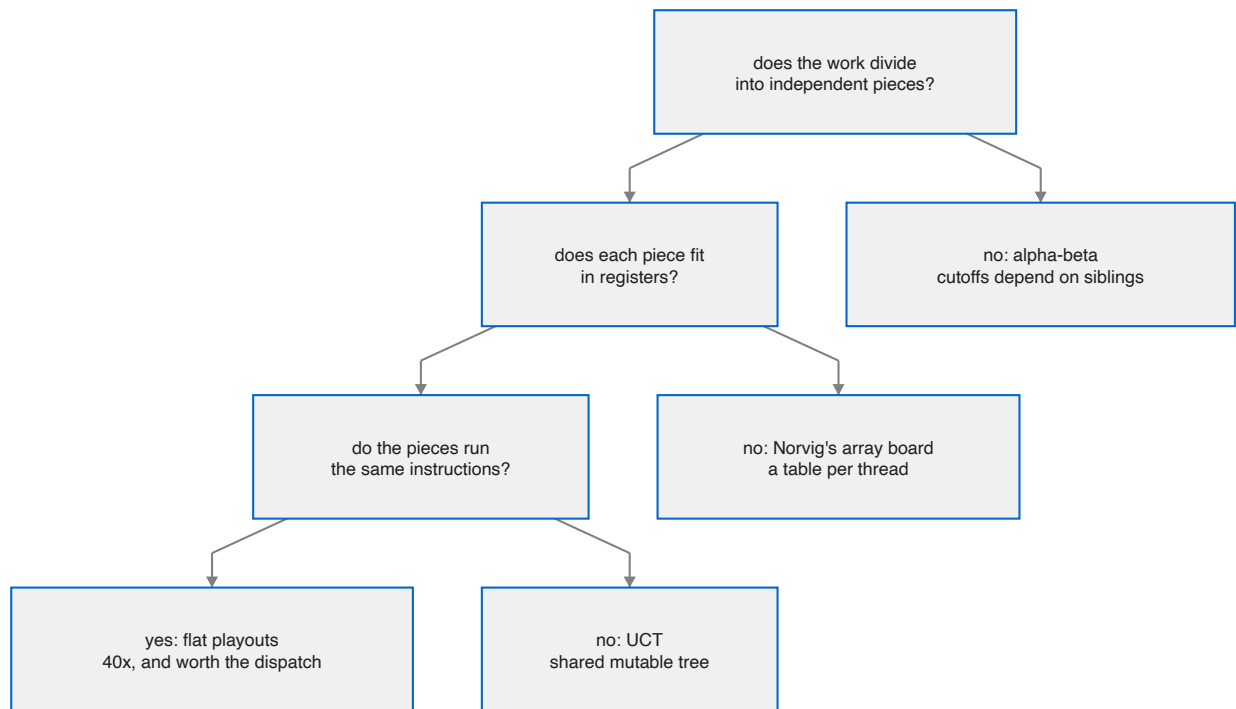
4. Because it is a completely different stress on the backend. Fluid is float-heavy, memory-bound, and 35 tiny dependent dispatches per frame. Othello is **integer-only**, register-resident, and **one large dispatch per move**. Same AIR backend, opposite corner of its behaviour — and it found a real compiler bug there ([chapter 6](#), the `llvm.scmp` story).

5. Because the fallback is honest. With no Metal device, Master runs the same playouts on the CPU with fewer of them, and the status bar says `Master · CPU`. The GPU is an accelerator for a level, not a level of its own.

6. Because trying to have both was worth the failure. The hybrid in `spikes/othello-mcts/` puts the tree on the CPU and the evaluations on the GPU. It loses to the flat player 0–6, and it is committed anyway, in `spikes/` and not `examples/`, because four of its bugs are worth keeping. [Chapter 7](#) is that story.

The summary the example would give

The short answer is that half of it does, and the useful part of this example is which half — and why.



3. The board is two integers

`board.mojo` is 121 lines and contains the entire ruleset of Othello. There is no board class, no square type, no list of pieces. There is this:

```
var black: UInt64
var white: UInt64
```

Bit i is row $i // 8$, column $i \% 8$, row 0 at the top — and the comment says why that particular convention:

the order the board is drawn in, so the UI never has to flip.

A small thing, but it is the sort of decision that saves a bug: Fluid has to flip its mouse coordinates because Cocoa's origin is bottom-left, and every flip is somewhere a sign can go wrong. Here the drawing code does the flip once, in one place, and the rules never think about it.

shift — one step, with the edge masked off

```
comptime NOT_LEFT = UInt64(0xFEFEFEFEFEFEFEFE) # every column but the first
comptime NOT_RIGHT = UInt64(0x7F7F7F7F7F7F7F7F) # every column but the last

fn shift(b: UInt64, dir: Int) -> UInt64:
  if dir == DIR_E:
    return (b << 1) & NOT_LEFT
  ...
```

Shifting a bitboard east by one moves every disc one column right — including bit 7, the last column, which lands on bit 8: the *first* column of the next row. That is a move through the edge of the board.

The mask is the fix, and the docstring names exactly what it replaces:

Masking after the shift is what the sentinel border was for.

Norvig's 100-element array carried 36 sentinel squares so that a walk off the edge would hit a wall. Here it is two constants, applied after the shift, costing one AND instruction. North and south need no mask at all — shifting by 8 pushes discs off the end of the word, which is exactly right.

legal_moves — every move at once

```
fn legal_moves(own: UInt64, opp: UInt64) -> UInt64:
  var empty = ~(own | opp)
  var moves = UInt64(0)
  for dir in range(8):
    var run = shift(own, dir) & opp
    for _ in range(5):
```

```

    run |= shift(run, dir) & opp
    moves |= shift(run, dir) & empty
return moves

```

Read it as a sentence. *Start from every square adjacent to one of mine that holds an opponent disc* (`shift(own, dir) & opp`). *Grow that run along the direction as long as it keeps hitting opponent discs* (the inner loop). *Wherever the run's far end is an empty square, that square is a legal move.*

Two details are worth pausing on.

The five. The docstring explains it exactly:

The run is grown six times because a line of eight squares can hold at most six discs between the two ends.

One initial step plus five in the loop. It is not a loop-until-done — it is a fixed count derived from the geometry of the board, so there is no data-dependent iteration count. **Every call executes precisely the same instructions.** On the CPU that is merely predictable. In a kernel where forty thousand threads run this in lockstep, it is the difference between the algorithm working and the algorithm being pointless.

No branch depends on the position. `dir` is a loop counter, the inner count is a constant, and everything else is `&`, `|`, `<<`, `>>`. Nothing in here asks "what is on this square?" — which is what makes it uniform.

The result is a bitboard, not a list. Every legal move in one word, ready to be iterated with `x &= x - 1` or counted with `popcount`.

flips_for — and the one place a branch survives

```

fn flips_for(own: UInt64, opp: UInt64, move: UInt64) -> UInt64:
    var flipped = UInt64(0)
    for dir in range(8):
        var run = shift(move, dir) & opp
        for _ in range(5):
            run |= shift(run, dir) & opp
        # The run only counts if one of ours closes it. Anything else -- an
        # empty square, or the edge -- and nothing turns over.
        if (shift(run, dir) & own) != 0:
            flipped |= run
    return flipped

```

Same propagation, run from the *move* rather than from your own discs. The difference is the test at the end: a run of opponent discs only flips if one of yours closes it. A run that ends at an empty square, or at the edge, flips nothing.

That `if` is the only position-dependent branch in the file, and it is harmless — it is a predicated `|=` that any compiler will turn into a select.

The docstring also states the function's contract in seven words: *"Empty if the move is not legal."* Which means callers do not need to check legality separately, and `apply_move` in `main.mojo` uses exactly that:

```
let f = flips_for(own, opp, move)
if f == 0:
    return
```

An illegal click is a move that flips nothing, and it is rejected by the same code that computes the flips.

popcount — and a deliberate refusal to be clever

```
fn popcount(b: UInt64) -> Int:
    """How many discs. The obvious loop, not the clever one: it runs once per
    move on the CPU and once per finished game on the GPU."""
    var n = 0
    var x = b
    while x != 0:
        x &= x - 1
        n += 1
    return n
```

`x &= x - 1` clears the lowest set bit — Kernighan's trick — so the loop runs once per disc rather than 64 times.

The docstring is doing something more interesting than describing the code: it is **justifying the absence of an optimisation**. There is a well-known branch-free SWAR popcount, and there is a hardware instruction. Neither is used, because this runs once per move and once per finished game. The comment records that the trade was considered and declined, which is what stops the next reader from "fixing" it.

Note that this is the *one* function here with a data-dependent iteration count — it runs `popcount(b)` times. Inside the kernel it is called only at the very end of a playout, after roughly sixty moves of uniform work, so the divergence it introduces is a rounding error.

Why 121 lines is the headline

The whole ruleset — legality, flipping, passing, counting, the opening position — is 121 lines with no data structure more complicated than a `UInt64`. And it is *checked*, which is the part that matters for a move generator:

```
perft 1 = 4          perft 5 = 1396
perft 2 = 12         perft 6 = 8200
perft 3 = 56         perft 7 = 55092
```

Perft is the count of distinct positions reachable in exactly n plies. It is the standard correctness test for a move generator, because these numbers are published and a generator that is subtly wrong —

one direction masked incorrectly, one edge case in passing — produces a number that is subtly wrong too, while still looking like it works.

The commit message makes the point:

because a move generator that is subtly wrong still looks like it works.

The whole of `board.mojo` compiles to something a GPU thread can run without touching memory. That is the property [chapter 2](#) spends itself arguing for, and this file is where it is actually earned.

4. Four players and one kernel

`ai.mojo` is 380 lines and holds four ways to choose a move. Its header comment is the thesis of the whole example, stated before any code:

Three ways to choose a move, and an honest account of which one wants a GPU.

Level	Method	Where
Beginner	a uniformly random legal move	CPU
Intermediate	alpha-beta, 3 ply, plain weighted evaluation	CPU
Advanced	alpha-beta, 4 ply, weighted plus corner bonus	CPU
Master	4,096 random playouts per candidate move	GPU

The evaluation function

`weight_at` is Norvig's table, and it is stored as **a quarter of itself**:

```
var r = index // 8
var c = index % 8
if r > 3:
    r = 7 - r
if c > 3:
    c = 7 - c
```

The table is symmetric in both axes, so only a quarter of it is written down: fold the square into the top-left 4x4 and look it up there.

Sixteen values instead of sixty-four, and the symmetry is enforced by construction rather than by carefully typing the same numbers four times.

The values encode the one thing everybody knows about Othello:

square	value	why
corner	120	can never be flipped
diagonally beside a corner	-40	taking one usually hands the corner over
beside a corner, on an edge	-20	same problem, less acute
centre	3-15	almost worthless

Negative values for squares you *hold* is the counter-intuitive part, and it is correct: in Othello, owning a disc can be a liability.

`smart_weighted` adds one refinement, and explains its own limits:

```
score += 30 * popcount(own & CORNERS)
score -= 30 * popcount(opp & CORNERS)
```

A corner is already worth 120, but holding one changes the value of the squares beside it — they stop being liabilities once the corner behind them is settled. This is a cheap stand-in for real stability analysis.

The last sentence is the useful one. Real Othello engines compute *stability*: which discs can provably never be flipped for the rest of the game. That is a substantial piece of work. This is a constant times a popcount, and the comment says so rather than pretending otherwise.

negamax — alpha-beta, with Othello's two wrinkles

```
fn negamax(own, opp, depth, alpha_in, beta, smart) -> Int:
  var alpha = alpha_in
  let moves = legal_moves(own, opp)
  if moves == 0:
    if legal_moves(opp, own) == 0:
      # game over: count discs
      ...
    # Pass: the turn goes over, the depth does not.
    return -negamax(opp, own, depth, -beta, -alpha, smart)
  if depth == 0:
    return smart_weighted(own, opp) if smart else weighted(own, opp)
```

Three things a chess implementation would not have:

A pass does not consume depth. If it did, a forced sequence of passes would burn the search horizon and the engine would evaluate a position it never actually reached. One comment, one line, easy to get wrong.

The terminal score is not the table. When neither player can move, the game is decided by counting discs — and the docstring puts it memorably:

at the end a disc is worth exactly one disc.

A positional table is a *proxy* for winning. At the end of the game the proxy is not needed and would actively lie, because a position with terrible shape and 33 discs is a win.

The margin is carried in the score. `WIN + mine - theirs` rather than a flat `WIN`, so among won lines the engine prefers winning by more, and among lost ones it prefers losing by less. That is what makes the losing games in the 6–0 result close rather than collapses.

The loop itself is ordinary alpha-beta, and its cutoff is exactly the property that makes it wrong for a GPU:

```
if alpha >= beta:
  break
```

That one line is the whole of chapter 2's objection. Whether it fires depends on what the previous sibling returned. Forty thousand threads each hitting a `break` at a different moment is the definition of divergence.

playout — the function that runs on both machines

```
fn playout(black_in: UInt64, white_in: UInt64, black_to_move: Bool,
           seed: UInt64) -> Int:
    """Play to the end at random. Returns +1 if black wins, -1 white, 0 drawn.

    The same function runs on the CPU and inside the GPU kernel. It touches
    no memory, which is why ten thousand copies of it can run at once.
    """
```

Its entire state is four values: two boards, a turn flag, an RNG word. No allocation, no array, no lookup table.

The RNG is **xorshift64***, chosen for the same reason:

because a playout needs a stream of numbers and nothing else: no state beyond one register, no tables, and the same three lines on both sides of the machine.

```
fn next_random(state: UInt64) -> UInt64:
    var x = state
    x ^= x >> 12
    x ^= x << 25
    x ^= x >> 27
    return x
```

Three instructions. A Mersenne Twister would need 2.5 KB of state per thread, which for forty thousand threads is 100 MB of memory traffic on a workload whose entire selling point is that it touches no memory.

The move choice is likewise register-only: `popcount` the legal moves, take `rng % count`, and pick that bit with `nth_bit`, which clears low bits one at a time.

And the passing rule, which is where a subtle bug would live:

```
var passes = 0
while passes < 2:
    ...
    if moves == 0:
        passes += 1
        turn_black = not turn_black
        continue
    passes = 0
```

`passes` resets on any real move. Two passes in a row means neither player can move — game over. One pass does not.

The three-way compare, and the compiler bug it found

The last line of `playout` is the strangest code in the example, and it has a seven-line comment because it needs one:

```
# Written as arithmetic rather than a three-way branch on purpose. The
# obvious `if b > w: 1 elif w > b: -1 else: 0` is folded into LLVM's
# three-way compare intrinsic, and the Metal backend has no such
# instruction -- the kernel then fails to link with "Undefined symbols:
# llvm.scmp.i32.i64", which says nothing about what wrote it.
# `Int(b > w) - Int(w > b)` is ALSO folded: zext(a>b) - zext(b>a) is the
# canonical shape LLVM turns into scmp. This one is not that shape.
return Int(b > w) * 2 - Int(b != w)
```

Follow what happened:

1. The obvious three-way `if` is recognised by LLVM and folded into `llvm.scmp`, a single intrinsic for "compare and return -1, 0 or 1".
2. The Metal backend has no such instruction, so the kernel **fails to link** with a message naming an LLVM intrinsic and nothing about Othello.
3. The obvious workaround — `Int(b > w) - Int(w > b)` — does not help. That is `zext(a>b) - zext(b>a)`, the *canonical pattern* the fold matches. Writing the trick out by hand hands the optimiser exactly what it was looking for.
4. `Int(b > w) * 2 - Int(b != w)` computes the same function through a different shape, which the fold does not recognise.

Verify it: if `b > w` then `2 - 1 = 1`; if `w > b` then `0 - 1 = -1`; if equal then `0 - 0 = 0`.

This is the kind of thing you can only find by writing a kernel that is not floating-point graphics, and it is one of the reasons [chapter 2](#) counts "a different stress on the backend" as a real justification.

The kernel

```
comptime PLAYOUTS_PER_MOVE = 4096
comptime GPU_BLOCK = 256

def playout_kernel(wins, black, white, moves_lo, move_count,
                  black_to_move, seed):
    """One thread, one playout.

    Threads are laid out as (move, playout): thread i studies move
    i // PLAYOUTS_PER_MOVE, so a whole warp works on the same candidate and
    walks the same code with different dice.
    """
```

The thread layout is the design decision. Thread i studies move `i // 4096` — so consecutive threads, which is to say the threads that execute together, are all playing out **the same candidate move**. They start from an identical position and diverge only through their dice.

Laid out the other way — thread i studies move `i % move_count` — adjacent threads would start from different positions, and the warp would be simultaneously exploring several different games from move one. Same total work, worse behaviour.

Each thread applies its move, then seeds itself distinctly:

```
let r = playout(nb, nw, not mover_black,
               seed ^ (UInt64(idx) * 0x9E3779B97F4A7C15))
```

A distinct stream per thread. Mixing the index in avoids every thread replaying the same game, which is the only way this can be wrong and still look plausible.

That last clause is worth reading twice. If every thread shared a seed, all 4,096 playouts of a move would be the *same game*, the counts would still come back, the player would still pick a move, and it would be a statistically meaningless one. The bug has no symptom.

The constant is the golden-ratio odd multiplier used throughout hashing — multiplying the index by it, then XOR into the seed, decorrelates neighbouring thread indices into distant states.

The reduction is on the CPU, on purpose

```
wins[unsafe_offset=idx] = Int32(r) if mover_black else Int32(-r)
```

Each thread writes one integer. Nobody sums anything on the device:

an atomic per playout would serialise exactly the thing that is supposed to be parallel, and the reduction is 4096 additions per move on a CPU that is idle anyway.

Forty thousand atomic increments to the same handful of addresses would serialise the workload whose entire justification is that it does not serialise. The alternative — a proper tree reduction in the kernel — would work, and would add shared memory, barriers and a second dispatch to save a few thousand additions on a CPU that is blocked on `synchronize` regardless.

The sign flip is also here: `Int32(r) if mover_black else Int32(-r)`, so the score is always from the mover's point of view and higher is always better. One flip, in one place — the same discipline the MCTS spike states explicitly about `uct_child`.

The dispatch

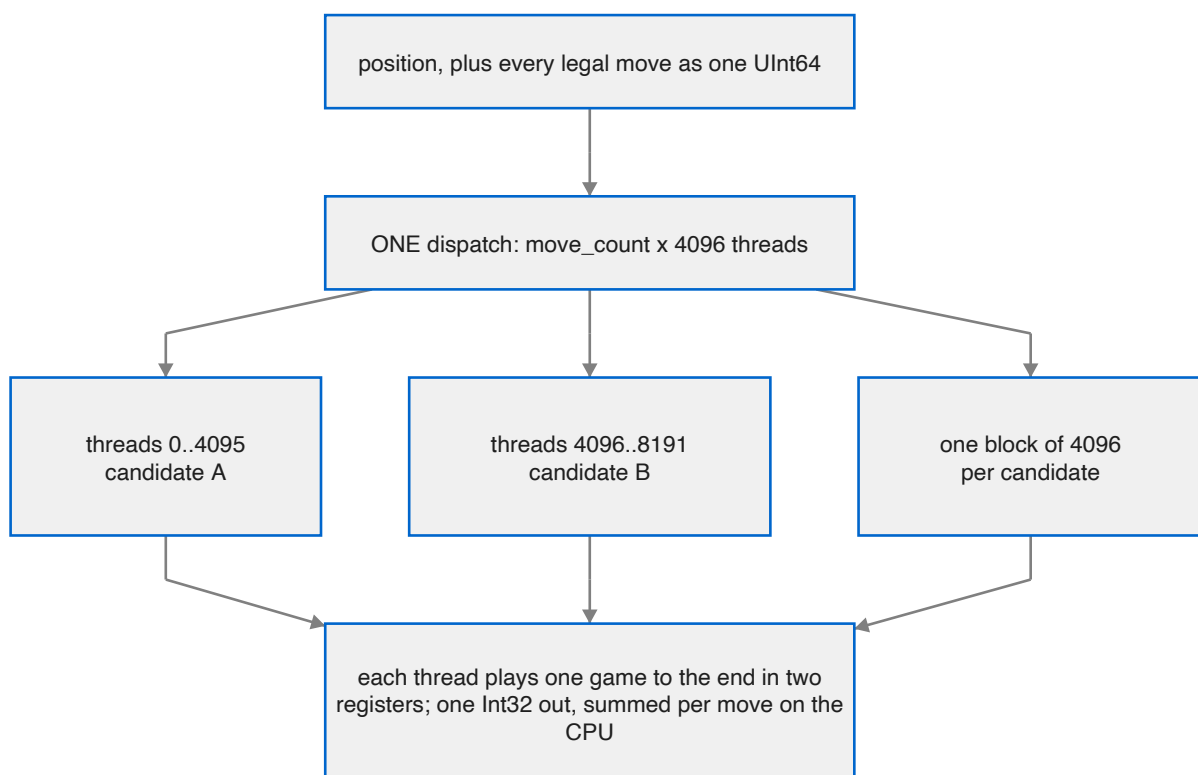
```
var kern = ctx.compile_function[playout_kernel]()
var results = ctx.enqueue_create_buffer[DType.int32](total)
let blocks = (total + GPU_BLOCK - 1) // GPU_BLOCK
```

```
ctx.enqueue_function(kern, results, black, white, moves, Int32(count), ..., seed,
                    grid_dim=(blocks), block_dim=(GPU_BLOCK))
ctx.synchronize()
```

One dispatch per move. Compare with Fluid's thirty-five per frame — the same backend, the opposite shape of workload. Here the launch overhead is paid once against milliseconds of work, which is why Othello never needed the async-launch investigation that Fluid motivated.

Note that the kernel is compiled and the buffer allocated **inside** the per-move function. In Fluid that would be indefensible; here a move takes 3.4 ms and happens when a human clicks, so a compile lookup per move is not worth the global to avoid it.

The whole legal-move set is passed as a single `UInt64` (`moves_lo`) and each thread walks to its own move with `m &= m - 1`. No move list, no index array — the argument is the data structure.



5. The window and the pump

`main.mojo` is 499 lines: a window, some drawing, and a loop. Almost all of its interesting decisions follow from one constraint, stated in the header:

The event loop is hand-rolled rather than `[NSApp run]` for the reason mandelbrot gives: a `DeviceContext` cannot live in a `named_global`, and a Cocoa callback cannot reach a local, so the loop that owns the GPU has to be the loop that drives the app. The view's handlers only set flags.

Why the loop is hand-rolled

Under `[NSApp run]`, AppKit owns the loop and your code runs from callbacks. That is the normal, correct way to write a Cocoa app, and here it does not work.

The obstruction is that a `DeviceContext` cannot be stored in a `named_global`. Process globals in this dialect are zero-initialised, so anything with a destructor needs a one-element `List` to live in one — and a `DeviceContext` is not a thing you can conjure from zeroed memory.

So the context has to be a local. And a Cocoa callback — a method on a class, invoked by AppKit — has no way to reach a local in `main`. The two facts together leave exactly one arrangement:

the loop that owns the GPU has to be the loop that drives the app.

`main` keeps the `DeviceContext` in a `var`, pumps the event queue itself, and handles everything. The view's handlers do not compute moves, do not touch Metal, and do not redraw. They set integers.

```
def mouseDown_(self, event: ObjCObject):
    # Only a flag: the pump owns the game, and the GPU, and the redraw.
```

This is the same shape Fluid uses, arrived at from the same constraint — and it is worth recognising as a *pattern* rather than a workaround. Anything that owns a GPU context in this dialect ends up driving its own loop.

The state, and why it is in globals

```
comptime g_black = named_global["oth.black", Int]
comptime g_white = named_global["oth.white", Int]
comptime g_black_turn = named_global["oth.turn", Int]
comptime g_level = named_global["oth.level", Int]
...
comptime g_click = named_global["oth.click", Int]      # 1 + square, or 0
comptime g_cmd = named_global["oth.cmd", Int]
```

The game, in globals, because the view's handlers and the pump both need it and neither can pass anything to the other.

Note `g_click` holds **1 + square**, not the square. Zero has to mean "nothing clicked", and square 0 is a real square (a1). The offset is what keeps the sentinel distinct from a legal value — the sort of detail that produces a board where the top-left corner mysteriously cannot be played.

Note also that the boards are `Int` here and `UInt64` everywhere else, with `board_black()` / `set_board()` converting at the boundary. The globals are storage; the rules get the right type.

The bug that taught the example something

This is the most valuable twenty lines in the file, and they are a comment:

```
# Read the flag out BEFORE clearing it. `let` binds by reference
# here, so `let cmd = g_cmd()[ ]` is a live view of the global and
# not a snapshot: clearing the global first makes every later
# read of `cmd` return zero, and the command silently evaporates.
```

The original code was the shape everybody writes:

```
let clicked = g_click()[ ]
if clicked != 0:
    g_click()[ ] = 0          # <- clicked is a REFERENCE, not a copy
    let m = bit(clicked - 1) # <- bit(-1)
```

In this dialect `let` **binds to a place, not a copy**. `clicked` is another name for the global, so clearing the global empties it, and `bit(-1)` is computed from a value that was correct one line earlier.

What makes this worth a chapter rather than a footnote is how it presented. From the commit message:

Every observable signal said the mouse worked: the handler ran, the coordinates were right, the square was legal. The failure was one line later, in code that reads like an obvious snapshot-then-clear.

Every diagnostic you would reach for says the code works. The handler fires. The coordinates map to exactly the four legal opening squares. The square is legal. And nothing happens — because the only thing wrong is a read that occurs after a write nobody thinks of as a write.

The fix is to derive what you need *before* clearing:

```
if g_cmd()[ ] != 0:
    let quit = (g_cmd()[ ] & CMD_QUIT) != 0
    let fresh = (g_cmd()[ ] & CMD_NEW) != 0
    g_cmd()[ ] = 0
```

Test the global directly, extract what you need, *then* clear. The same pattern is applied to `g_click` immediately below.

The compiler now warns about the tracked form of this — "*this write changes what 't' reads*" — but it cannot see through the untracked-origin pointer a `named_global` hands back, which is exactly the

case here.

The other bug in the same commit

The same commit fixed two more, and they had a common cause:

An NSPoint is two doubles returned in registers, and that path does not describe the return shape to the ABI, so the call yields nothing usable and every click computes the same wrong square — with no error anywhere.

The click point was read through the dynamic path, which does not tell the ABI that the return is a two-double struct. The keyboard had it too, reading `charactersIgnoringModifiers`, so N, Q and the level keys were all dead.

Both are now typed calls, and the mouse handler carries a note that reads as history rather than warning:

```
# The point comes back typed: locationInWindow's @encode says
# NSPoint, and the call path maps that to CGPoint through the kind
# ladder -- two doubles in registers, described to the ABI by the
# database rather than by anyone's memory of it. (It was not always
# so: the hand-typed msg_send here used to be the only safe shape.)
```

Three bugs in one commit, and all three shared a property: **the program looked like it was working**. Nothing raised, nothing crashed, no diagnostic fired. That is what makes them worth writing down.

Drawing

Deliberately plain. No layers, no Core Animation, no image caching — the whole board is redrawn on demand:

```
class OthelloView(NSView):
    def drawRect_(self, dirty: CGRect):
        draw_board()
        draw_status()
```

Grid lines are drawn as thin rectangles rather than paths:

a board this size needs nine of them each way and no path object.

Discs are `NSBezierPath` ovals. Legal moves for the human are drawn as a small dot:

Small, so it reads as advice rather than as a disc already placed.

And the one coordinate flip in the program, isolated to the drawing code:

```
# Row 0 is the top of the board; the view is not flipped, so
```

```
# the top row is the highest y.
let y = MARGIN + STATUS_H + Float64(7 - row) * CELL
```

The rules use row 0 = top because that is how the board is drawn; the view uses Cocoa's bottom-left origin because that is what AppKit gives you. The `7 - row` reconciles them, once, here.

The pump

```
while running:
  while True:
    var ev = app.nextEventMatchingMask(UInt64.MAX,
                                      untilDate=ObjCObject(past.id), inMode=mode, dequeue=True)
    if ev.id == 0:
      break
    _ = app.sendEvent(ObjCObject(ev.id))
  if not win.isVisible():
    break
```

Drain the queue with `distantPast` — poll, do not block — dispatch each event so the view's handlers run, then act on the flags they set.

Note `_ = app.sendEvent(...)`: Fluid's pump reads events and interprets them directly, so it never needs to send them on. Here the view has real handlers, so events must be delivered.

Then, in order: commands, the human's move, the computer's move.

```
if g_over()[] == 0 and g_black_turn()[] == 0:
  g_thinking()[] = 1
  redraw()
  rng = next_random(rng)
  let t0 = perf_counter_ns()
  var move = UInt64(0)
  if g_level()[] == LEVEL_MASTER and have_gpu:
    move = best_by_playouts_gpu(ctx, board_black(), board_white(), False, rng)
  else:
    move = computer_move_cpu(rng)
  let t1 = perf_counter_ns()
  g_last_ms()[] = Int((t1 - t0) // 1000000)
```

This is the only place the `DeviceContext` is used, and it is a local in `main` — which is the constraint the whole architecture was built around.

The `g_thinking()` flag plus `redraw()` before the search is a nicety with a sharp edge: the pump blocks for the duration of the move, so the redraw has to be *requested and drawn* before the block, not after. On the GPU that block is 3.4 ms and nobody would see it. At `Advanced` on a slow machine, or Master falling back to CPU playouts, it matters.

The elapsed time goes into the status bar in milliseconds, which is how the numbers in [chapter 2](#) get taken: the program reports its own thinking time on every move.

The GPU probe

```
var have_gpu = False
var ctx = DeviceContext(api="metal")
try:
    _ = best_by_playouts_gpu(ctx, board_black(), board_white(), False, 1)
    have_gpu = True
except:
    have_gpu = False
```

The GPU, if this machine has one. Master falls back to CPU playouts if not, which is worth doing rather than hiding the level.

Not a capability query — an actual playout dispatch on the opening position, discarded. The only reliable way to know whether the kernel compiles, links and runs on this machine is to compile, link and run it. Any query short of that would have missed the `llvm.scmp` link failure from [chapter 4](#).

The result is visible rather than hidden: the level shows as `Master · GPU` or `Master · CPU` in the status bar, and the CPU fallback plays the same algorithm with 512 playouts per move instead of 4,096.

6. What to understand

Eight things about this example are not obvious from playing it.

1. The example's answer is "no", and that is the deliverable

It would have been easy to write a GPU Othello and present it as a success. The example instead ships alpha-beta on the CPU, measures it at 81 μ s, and says that moving it would be slower. From the source:

Moving that to a GPU would make it slower, and saying otherwise would be a demo rather than an argument.

The one place the GPU *does* help is genuinely narrow: uniform, stateless, register-resident Monte-Carlo playouts. Everything else about a board game resists the hardware. If you take one thing away, take the shape of the test rather than the verdict — *independent, register-sized, uniform control flow* is the question to ask of any workload.

2. The bitboard is load-bearing, not a flourish

Two `UInt64`s instead of Norvig's 100-element array is not a speed tweak. It is the reason a GPU thread can hold an entire game:

a thread that plays a whole game needs two registers for the board, and every thread executes the same instructions regardless of what its game looks like.

With an array board, each of forty thousand threads would need its own 100 bytes in memory, and the workload whose entire justification is "*it touches no memory*" would touch four megabytes of it per dispatch. The representation is what makes the argument true.

3. `legal_moves` has no data-dependent loop

```
for dir in range(8):
    var run = shift(own, dir) & opp
    for _ in range(5):
        run |= shift(run, dir) & opp
```

Eight, then five. Both constants, derived from the geometry: a line of eight squares holds at most six discs between its ends. **No branch in the whole function depends on what is on the board.**

On a CPU that is merely nice. In a warp of threads playing forty thousand different games, it is the difference between the algorithm working and being pointless — every thread runs the identical instruction sequence, and only the bits differ.

4. Every thread must have its own dice, and the bug has no symptom

```
seed ^ (UInt64(idx) * 0x9E3779B97F4A7C15)
```

Mixing the index in avoids every thread replaying the same game, which is the only way this can be wrong and still look plausible.

Share a seed across threads and all 4,096 payouts of a move are the *same game*. The counts still come back. The player still picks a move. It picks it from a sample of size one, disguised as a sample of size 4,096, and no diagnostic anywhere will tell you.

5. The reduction is on the CPU, and that is the right call

Each thread writes one `Int32` to its own slot. Nothing is summed on the device:

an atomic per payout would serialise exactly the thing that is supposed to be parallel, and the reduction is 4096 additions per move on a CPU that is idle anyway.

Forty thousand atomics onto a handful of addresses would serialise the workload whose only virtue is that it does not serialise. A proper tree reduction would work and would cost shared memory, barriers and a second dispatch — to save a few thousand additions on a CPU that is blocked on `synchronize` anyway.

6. Two ways to compute the same function are not the same to a compiler

```
return Int(b > w) * 2 - Int(b != w)
```

The obvious `if b > w: 1 elif w > b: -1 else: 0` is folded into `llvm.scmp`, which the Metal backend cannot lower — so the kernel fails to link with a message naming an intrinsic and nothing else. And the obvious workaround, `Int(b > w) - Int(w > b)`, is folded too: it is the *canonical shape* the optimiser looks for.

Two lessons. A link error naming an LLVM intrinsic is usually a lowering gap, not your bug. And writing an idiom out by hand can hand the optimiser exactly the pattern you were trying to avoid.

7. let binds to a place, and the failure is invisible

```
let clicked = g_click()[ ] # a live view of the global, not a snapshot
if clicked != 0:
```

```
g_click()[ ] = 0          # clicked is now 0
let m = bit(clicked - 1)  # bit(-1)
```

The handler fires, the coordinates are right, the square is legal, and the move vanishes one line later. Read the value **out** of a global before clearing it — test it directly, derive what you need, then reset.

8. Speed bought strength, not just speed

The 40x is the headline and the least interesting number here. What it actually bought is a *different player*:

```
Master (GPU playouts)  6 - 0  Advanced (alpha-beta, 4 ply)
```

Six games, margins of 6 to 26 discs, all played in half a second. A level that would have stalled for most of a second per move can now answer immediately, so it can afford to be the strongest thing in the app. That crude-but-plentiful beats sophisticated-but-shallow here is a real result — and [chapter 7](#) is what happened when we tried to have both.

A short list of things that will bite

If you change...	...this happens
the seed to be shared across threads	4,096 identical games, reported as 4,096 samples
the thread layout to <code>idx % move_count</code>	a warp explores several different games at once
the inner <code>range(5)</code> to a while-until-done	data-dependent iteration; the warp diverges
the reduction into the kernel as atomics	serialises the one part that was parallel
<code>Int(b>w)*2 - Int(b!=w)</code> back to an <code>if</code>	the kernel stops linking, citing <code>llvm.scmp</code>
<code>var</code> to <code>let</code> around a global you clear	the value evaporates, with no symptom
a pass to consume depth in negamax	the search horizon burns on forced sequences
the terminal score to use the table	positions won on discs are evaluated as losses

Controls

click	play a disc — legal squares are dotted
N	new game
B / I / A / M	Beginner, Intermediate, Advanced, Master
Q or Esc	quit

The status bar shows the disc count, the level, and the time the computer's last move took — which is where every number in [chapter 2](#) comes from.

7. The sequel that does not work

`spikes/othello-mcts/` is the obvious next move, and its README opens by admitting the obvious next move failed:

Hybrid UCT + GPU payouts — unfinished

It does not work yet: it loses to the flat Monte-Carlo player in `examples/othello/` 0–6, and more search makes it worse.

It is in `spikes/` rather than `examples/` deliberately — “the shipped example should be a thing that works” — and it is committed anyway, because four of its bugs are worth keeping. This chapter is the account.

The idea, and why it should have worked

[Chapter 2](#) leaves an unsatisfying result: the GPU only accepts the *weakest* member of the Monte-Carlo family. UCT is far stronger per sample, and it is GPU-hostile for exactly alpha-beta's reason — the tree is shared mutable state and each sample depends on the last.

So split the work along the line where it actually divides:

CPU	owns the tree — selection, expansion, backpropagation. Sequential, branchy, tiny.
GPU	answers one question — “how good is this position?” — for 64 positions at once.

The economics are compelling:

A kernel launch costs the same whether it evaluates one leaf or sixty-four, so the tree nominates a batch before each dispatch.

That is **leaf parallelisation**, a known and respectable technique, and **virtual loss** is what stops all 64 nominations picking the same node — each outstanding nomination counts as a temporary loss, pushing the next one elsewhere.

The kernel is the flat one, generalised from one position to an array:

```
def batch_payout_kernel(results, blacks, whites, turns,
                        position_count, per_position, seed):
    let which = idx // Int(per_position)
    let r = payout(blacks[unsafe_offset=which], whites[unsafe_offset=which],
                  turns[unsafe_offset=which] != 0,
                  seed ^ (UInt64(idx) * 0x9E3779B97F4A7C15))
    # Always from black's point of view. The tree flips the sign where it
    # needs to, once, rather than every thread guessing whose turn it is.
    results[unsafe_offset=idx] = Int32(r)
```

64 leaves × 32 payouts = 2,048 threads per dispatch.

Four bugs, each invisible in a different way

This is the part worth keeping, and the README is unusually precise about each. All four are **scale and ordering errors**, not logic errors — the code did what it said, in the wrong units or at the wrong moment.

1. The descent never stopped

```
t_pending()[][node] += 1
# The test has to come AFTER the increment above -- testing before it
# never fires, and the descent then runs all the way to a finished game
if node != 0 and t_visits()[][node] == 0 and t_pending()[][node] == 1:
    return node
```

The "is this node newly nominated?" test ran *before* the counter it reads was incremented, so it never fired. Every selection descended to a finished game, expanding the whole line on the way.

60 selections built 7,256 nodes instead of 129, and every evaluation was of a position whose result was already decided.

Fifty-six times too many nodes, and every measurement taken of a position with nothing left to measure.

2. Virtual loss in the wrong units

`pending` counted **nominations** (1 each) and was compared against `visits`, which counts **playouts** (192 each in the original configuration). The loss was numerically invisible, so all 64 leaves in a batch followed one path — the precise failure virtual loss exists to prevent.

3. Exploration scaled to samples, not decisions

UCT's exploration term `sqrt(log N / n)` is written for a count of **decisions**. A nomination here brings back `PER_LEAF` playouts at once, so counting `visits` directly makes the term about `sqrt(PER_LEAF)` too small.

exploration vanished and the search went greedy, putting 89% of its samples on one move in a balanced opening.

The fix is in the comment, and it is a nice distinction:

The mean is over playouts, which is the honest estimate; the exploration term is over nominations, which is the right scale.

4. The root move chosen by visit count

Most-visited is the standard MCTS criterion, and it is right *when the search is long enough for the best move to pull clearly ahead*. At a few thousand nominations the root children's visit counts are within noise of one another, so "most visited" picks whichever happened to be nominated marginally more often. Changed to choose by mean, which is computed over tens of thousands of playouts.

After all four, it looks correct — and still loses

visits spread 66k–78k over four near-equal opening moves, and the means agree with direct sampling to within 0.01.

Every diagnostic says the search is healthy. And:

```
16 rounds    → loses by  8, 24, 14, 34, 38, 48
128 rounds   → loses by 48, 44, 40, 30, 32, 54
```

More search makes it worse. That is the diagnostic that matters, and the README draws the right conclusion from it:

A correct search that is merely under-resourced improves with more simulations. One that degrades is converging confidently on the wrong quantity, so the remaining defect is in what gets backed up, not in tuning.

This is a genuinely useful piece of reasoning. Monotonicity in resources is a *property* you can test, and its violation cleanly separates "needs tuning" from "is measuring the wrong thing" — without knowing where the bug is.

The two remaining suspects are named:

- the mean at a node mixes playouts taken at different depths and under different amounts of tree guidance — normal for MCTS, and it makes a sign or perspective error very hard to see from the root
- `t_score` is black-positive everywhere and the flip happens once, in `uct_child` — the right design, and *"exactly where a single misplaced negation would produce this symptom"*

And the experiment that would settle it: instrument one move end to end, dump the principal variation, and check by hand whether the backed-up value at depth 2–3 matches a direct playout estimate of that same position. That isolates backup from selection, which is the split the evidence has not yet cut.

Two incidental things worth knowing

A struct of ten Lists crashes the compiler. The tree is parallel arrays in module globals rather than a struct, and not by preference:

A struct with ten List fields crashes this compiler outright — incorrect # of replacement values out of DialectConversion — before any of this code runs.

`_log` and `_sqrt` are written out by hand. Newton's method for the root, and an atanh series for the log after halving into $[1, 2)$ — *"the `stdlib` has one; this file avoids the `import` so the same arithmetic runs everywhere."* The same instinct as Fluid's hand-written `_expf`: a function that must be identical on host and device is written once, in a form that compiles for both.

What the failure is worth

The flat player on the GPU wins because it is crude enough to parallelise perfectly. The tree player should beat it on samples and does not, because the plumbing between a sequential tree and a batched evaluator has four places to get the *units* wrong and one more, still unfound, to get the *sign* wrong.

That is a real result about the hybrid pattern, and it is why the spike is committed rather than deleted. It also sharpens [chapter 2](#)'s conclusion: the GPU did not merely select a weaker algorithm — it made the stronger one *harder to get right*, and that cost is not visible until you try.

